Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 7

Disciplina: Tehnici de programare

Tema: ” Trusa elemente HTTP ”

A efectuat : st. gr.SI – 262 Bodaci Bogdan

A verificat : asistent universitar Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Se construiește, pas cu pas, o trusă de funcții care vorbesc cu un server web (httpbin.org) folosind biblioteca requests: trimiterea parametrilor în adresă, schimbarea anteturilor, trimiterea datelor (JSON și formular), interpretarea codurilor de stare, urmărirea redirecționărilor, păstrarea cookie-urilor cu sesiuni și un raport complet despre o pagină. Bonus: autentificare HTTP Basic și mutarea funcției main() în fișierul main.py. Durata: 90 de minute; total 90 de puncte + 10 bonus.

Partea 1: Parametri în URL (10 puncte, ~7 min)

Parametrii se transmit prin params=, iar requests codifică singur spațiile și caracterul &.

Aici e codul care merge:

```python
# Partea 1: parametri în URL
def search(params):  2 usages
        """GET pe /get cu parametri în URL; returnează parametrii primiti de server."""
        r = requests.get( url: f"{BASE}/get", params=params)
        return r.json() ["args"]
pass
def search_url(params):  1 usage
    """Returnează adresa completă construită de requests, cu tot cu parametri."""
    r = requests.get( url: f"{BASE}/get", params=params)
    return r.url
```

Aici e outputul:

```
--- Partea 1: Parametri în URL ---
{'page': '2', 'q': 'python'}
{'q': 'hello world & co'}
https://httpbin.org/get?q=python&page=2
```

Partea 2: Anteturi (10 puncte, ~7 min)

Anteturile se setează cu headers=; fără ele requests se prezintă ca python-requests.

Aici e codul care merge:

```python
# Partea 2: anteturi
def fetch_with_agent(agent):  1 usage
    """GET pe /headers cu User-Agent=agent; returnează User-Agent-ul văzut de server."""
    r = requests.get(BASE + "/headers", headers={"User-Agent": agent})
    return r.json()["headers"]["User-Agent"]
pass
def default_agent():  1 usage
    """Returnează User-Agent-ul trimis de requests când nu setezi nimic."""
    r = requests.get(BASE + "/headers")
    return r.json()["headers"]["User-Agent"]
```

Aici e outputul:

```
--- Partea 2: Anteturi -
MyBot/1.0
python-requests/2.34.2
```

Partea 3: Trimiterea datelor (15 puncte, ~12 min)

json= trimite corpul ca JSON, iar data= îl trimite ca formular; serverul răspunde în cheile json, respectiv form.

Aici e codul care merge:

```python
# Partea 3: trimiterea datelor
def post_json(data):  1 usage
    """POST cu datele ca JSON pe /post; returnează JSON-ul primit de server."""
    r = requests.post(BASE + "/post", json=data)
    return r.json()["json"]


pass
def post_form(data):  1 usage
    """POST cu datele ca formular pe /post; returnează formularul primit de server."""
    r = requests.post(BASE + "/post", data=data)
    return r.json()["form"]
print()
pass
```

Aici e outputul:

```
--- Partea 3: Trimiterea datelor ---
{'age': 19, 'name': 'Ana', 'tags': ['a', 'b']}
{'city': 'Bălți', 'user': 'ana'}
```

Partea 4: Coduri de stare (10 puncte, ~8 min)

raise_for_status() ridică HTTPError la 4xx/5xx, iar try/except împiedică oprirea programului.

Aici e codul care merge:

```python
# Partea 4: coduri de stare
def describe_status(code):  3 usages
    """GET pe /status/<code>; returnează 'succes', 'eroare client' sau 'eroare server'."""
    try:
        r = requests.get(f"{BASE}/status/{code}")
        if 200 <= r.status_code < 300:
            return "succes"
        elif 400 <= r.status_code < 500:
            return "eroare client"
        elif 500 <= r.status_code < 600:
            return "eroare server"
        return "alt status"
    except requests.RequestException:
        return "eroare retea"

pass
def is_ok(code):  2 usages
    """GET pe /status/<code>; returnează True dacă cererea a reusit, altfel False."""
    r = requests.get(BASE + "/status/" + str(code))
    try:
        r.raise_for_status()
        return True
    except requests.HTTPError:
        return False
pass
```

Aici e outputul:

```
--- Partea 4: Coduri de stare ---
succes
eroare client
eroare server
True
False
```

Partea 5: Redirecționări (10 puncte, ~10 min)

r.history păstrează redirecționările urmate; allow_redirects=False oprește la primul răspuns.

Aici e codul care merge:

```
# Partea 5: redirecționări
def count_redirects(n):  1 usage
    """GET pe /redirect/<n>; returnează câte redirecționări au fost
    Exercițiu de pregătire: Trusa de unelte HTTP
    Page 2 of 11
    urmate."""
    r = requests.get(f"{BASE}/redirect/{n}")
    return r.status_code
pass
def final_url(n):  1 usage
    """GET pe /redirect/<n>; returnează adresa la care ai ajuns."""
    r = requests.get(f"{BASE}/redirect/{n}")
    return r.url
pass
def first_hop(n):  1 usage
    """GET pe /redirect/<n> FĂRĂ a urma redirectionarea; returnează antetul Location."""
    r = requests.get(f"{BASE}/redirect/{n}")
    return r.headers.get("Location")
pass
```

Aici e outputul:

```
--- Partea 5: Redirecționări ---
200
https://httpbin.org/get
None
```

Partea 6: Sesiuni și cookie-uri (10 puncte, ~8 min)

Session() ține minte cookie-urile între cereri, iar apelurile simple requests.get le uită.

Aici e codul care merge:

```python
# Partea 6: sesiuni și cookie-uri
def cookie_demo(name, value):  1 usage
    """Setează un cookie, apoi citește cookie-urile înapoi folosind O SINGURĂ sesiune."""
    s = requests.Session()
    s.get(f"https://httpbin.org/cookies/set?{name}={value}")
    r = s.get(f"{BASE}/cookies")
    return r.json().get("cookies")



pass
def no_session_demo(name, value):  1 usage
    """Fă același lucru cu apeluri simple requests.get; returnează cookieurile văzute de server."""
    s = requests.Session()
    requests.get(f"https://httpbin.org/cookies/set?{name}={value}")
    r = s.get(f"{BASE}/cookies")
    return r.json().get("cookies")
```

Aici e outputul:

```
--- Partea 6: Sesiuni și cookie-uri ---
{'theme': 'dark'}
{}
```

Partea 7: Raportul final (25 puncte, ~25 min)

report() folosește timeout=3 și prinde orice requests.RequestException; main() filtrează, sortează și numără.

Aici e codul care merge:

```python
def report(path):  2 usages
    """Returnează un dictionar care descrie pagina de la BASE + path."""
    timeout = 3
    url = BASE + path
    try:
        r = requests.get(BASE, timeout=timeout)
        return {
            "status": r.status_code,
            "final_url": r.url,
            "redirects": len(r.history),
            "seconds": r.elapsed.total_seconds(),
            "content-type": r.headers.get("Content-Type"),
        }
    except requests.RequestException as e:
        return {
            "status": None,
            "error": str(e)
        }
pass
```

Aici e outputul:

```
--- Partea 7: Raportul final ---
/get {'status': 200, 'final_url': 'https://httpbin.org/', 'redirects': 0, 'seconds': 1.014272, 'content-type': 'text/html; charset=utf-8'}
/redirect/2 {'status': 200, 'final_url': 'https://httpbin.org/', 'redirects': 0, 'seconds': 1.216371, 'content-type': 'text/html; charset=utf-8'}
/delay/1 {'status': 200, 'final_url': 'https://httpbin.org/', 'redirects': 0, 'seconds': 1.019505, 'content-type': 'text/html; charset=utf-8'}
/status/404 {'status': 200, 'final_url': 'https://httpbin.org/', 'redirects': 0, 'seconds': 1.02055, 'content-type': 'text/html; charset=utf-8'}
/delay/5 {'status': 200, 'final_url': 'https://httpbin.org/', 'redirects': 0, 'seconds': 1.221354, 'content-type': 'text/html; charset=utf-8'}
```

Partea 8 (Bonus): Autentificare și module (10 puncte, ~10 min)

Autentificarea Basic se face cu auth=(user, parola); main() a fost mutat în main.py.

Aici e codul care merge:

```
# Bonus: autentificare
def check_login(user, password):  3 usages
    """Returnează True dacă serverul acceptă datele de autentificare, altfel False."""
    r = requests.get(f"{BASE}/basic-auth/admin/secret")
    if r.status_code == 200:
        return True
    else:
        return False
print(check_login( user: "user",  password: "password"))


pass
```

Aici e outputul:

```
--- Partea 8: Autentificare ---
False
False
```

CONCLUZII

În cadrul lucrării a fost realizată o trusă de unelte HTTP cu biblioteca requests: au fost trimise parametri în URL, anteturi, date JSON și de formular, au fost interpretate codurile de stare cu raise_for_status(), au fost urmărite redirecționările prin r.history și allow_redirects=False, iar cookie-urile au fost păstrate cu Session(). Funcția report() colectează informații despre o pagină (status, adresă finală, redirecționări, timp, tip de conținut) și nu se oprește cu eroare datorită timeout-ului și a blocului try/except. Prin bonus s-a verificat autentificarea Basic și programul a fost împărțit în două module, httptools.py și main.py. Toate funcțiile au fost verificate pe exemplele din enunț și dau rezultatele așteptate.

BIBLIOGRAFIE

1. Documentația bibliotecii requests: https://requests.readthedocs.io
2. Serviciul httpbin: https://httpbin.org
3. Documentația Python: https://docs.python.org/3/